Below is a production-focused research report based only on official Toast documentation and Microsoft Learn where Fabric design choices matter. The core conclusion is this: for a finance/reporting warehouse, the **Toast Standard API is the primary system of record** because it exposes transaction-level data across orders, checks, selections, payments, labor, cash, restaurant metadata, menus, and configuration. The **Analytics API is separate, Pro-gated, management-group oriented, and Toast explicitly says its metrics data does not adhere to GAAP/accounting standards**. (doc.toasttab.com)

1. Authentication and access

How Toast Standard API machine-client authentication works

Toast uses the OAuth 2.0 **client credentials** flow. You request a token by POSTing to /authentication/v1/authentication/login with JSON containing clientId, clientSecret, and userAccessType: "TOAST_MACHINE_CLIENT". The authentication response returns a bearer token and expiresIn, and the token itself encodes claims including expiration and scopes. (doc.toasttab.com)

Required headers

For the auth call, use Content-Type: application/json. For secured API calls, Toast requires Authorization: Bearer <token> and the location context header Toast-Restaurant-External-ID: <restaurant GUID>. Toast's docs are explicit that Standard API requests are **location-specific**, so you make separate requests per restaurant location. (doc.toasttab.com)

Token refresh / expiration behavior

Tokens are generally valid for about one day. Multiple auth requests during the same validity window return the **same token** and same expiration. Toast recommends reusing tokens as long as possible, requesting no more than one or two tokens per day, and not more than twice in an hour. A new token should be requested in the last minute before expiry or after expiry. (doc.toasttab.com)

Scope requirements

Standard API scopes relevant to warehousing/reporting are:

- orders:read
- config:read
- menus:read
- restaurants:read

- labor:read
- labor.employees:read
- cashmgmt:read
- optionally guest.pi:read
- optionally delivery_info.address:read (doc.toasttab.com)

Toast's own warehouse cookbook lists cashmgmt:read, config:read, labor.employees:read, labor:read, menus:read, restaurants:read, and orders:read as the required base scope set for a data warehouse integration. If you want guest names/contact info or delivery addresses, you also need guest.pi:read and delivery_info.address:read. (doc.toasttab.com)

Restaurant GUID usage

When credentials are created, Toast sends an email with the selected location names and GUIDs. Toast states that these GUIDs are the values you use in requests and that they are also referred to as Toast-Restaurant-External-ID or restaurantGuid. For management-group discovery, the restaurants API provides /restaurants/v1/groups/{managementGroupGUID}/restaurants, which returns restaurant GUIDs. (doc.toasttab.com)

Webhook setup requirements

Webhook subscriptions are **maintained by Toast support**. Each subscription has its own webhook secret, unique per subscription and environment. Your endpoint must be publicly accessible, Toast recommends validating messages with HMAC-SHA256 signing, and your endpoint must return a 2xx acknowledgment within **2 seconds** before doing downstream processing. Toast retries on timeout, 404, 429, and 5xx; it does not retry on other 4xx or 3xx responses. (doc.toasttab.com)

Rate limits and important restrictions

Toast's global and default API rate limits are **20 requests/second** and **10,000 requests/15 minutes**. Two important endpoint-specific limits matter here: GET /menus is **1 request/second per location**, and GET /ordersBulk is **5 requests/second per client per location**. For historical /ordersBulk retrieval, Toast says windows must not exceed one month and calls should be spaced **5–10 seconds apart**. Standard API access is read-only, location-specific, supports **GET only**, does **not** include sandbox access, and only supports **Menus V2**, not Menus V3. (doc.toasttab.com)

2. Reporting-relevant Standard API coverage

This is the relevant Standard-API surface for finance, operations, and warehousing. Toast's own warehouse/sales/labor/cash cookbooks point to these as the core retrieval surfaces for reporting. (doc.toasttab.com)

Core transaction and master-data endpoints

Domain	Endpoint / object	Purpose	Required scope	Grain	Warehouse suitability	Key fields / notes	Major limitations
Orders	GET /orders/v2/ordersBulk (Toast API reference documentation)	Bulk retrieval of full order JSON by startDate/endDate or businessDate	orders:read (doc.toasttab.com)	One Order per row, nested checks/selections/payments	**Highest-priority** source for transaction warehousing	guid, businessDate, openedDate, modifiedDate, promisedDate, revenueCenter, server, diningOption, nested checks[] (Toast API reference documentation)	Nested JSON; per-location; businessDate only returns orders created that day, not older orders modified that day; use startDate/endDate for change capture (Toast

Domain	Endpoint / object	Purpose	Required scope	Grain	Warehouse suitability	Key fields / notes	Major limitations
							API reference documentation)
Order detail	GET /orders/v2/orders/{guid} (Toast API reference documentation)	Single-order reread / targeted repair	orders:read (doc.toasttab.com)	One order	Good for replay/repair, not main bulk extractor	Full current order state	Not efficient for primary backfill
Payments	GET /orders/v2/payments (/payments) (doc.toasttab.com)	All payments by paidBusinessDate, refundBusinessDate, voidBusinessDate	orders:read (doc.toasttab.com)	One payment	**Required** for finance-grade cash/card/refund/void reporting	amount, tipAmount, type, paidBusinessDate, refund, voidInfo, server, cashDrawer, otherPayment, houseAccount, cardPaymentId, tenderTransactionGuid (Toast	Must be pulled separately; Toast recommends retrieving at least six hours after closeoutHour to capture all

Domain	Endpoint / object	Purpose	Required scope	Grain	Warehouse suitability	Key fields / notes	Major limitations
						API reference documentation)	CC payments and fees (doc.toasttab.com)
Payment detail	GET /orders/v2/payments/{guid} (doc.toasttab.com)	Retrieve one payment by GUID	orders:read	One payment	Useful for replay/debugging	Detailed Payment object	Not a primary bulk extractor
Employees	GET /labor/v1/employees (doc.toasttab.com)	Employee dimension / server mapping	labor.employees:read (doc.toasttab.com)	One employee	Essential dimension	guid, firstName, lastName, chosenName, externalEmployeeId, email, jobReferences, deleted, disabled (doc.toasttab.com)	Current-state employee master, not historical SCD by itself
Time entries	GET /labor/v1/timeEntries (Toast API reference documentation)	Labor/payroll-like transact	labor:read	One time entry	Core for labor reporting	guid, employeeReference, jobReference,	Values can change until clock-

Domain	Endpoint / object	Purpose	Required scope	Grain	Warehouse suitability	Key fields / notes	Major limitations
		ion retrieval				shiftReference, createdDate, modifiedDate, deleted, breaks; supports businessDate, start/end, and modifiedStart/modifiedEnd filters (Toast API reference documentation)	out; completed entries then become static (doc.toasttab.com)
Shifts	GET /labor/v1/shifts (doc.toasttab.com)	Scheduled shifts vs actual work	labor:read	One scheduled shift	Optional but useful for ops variance	Scheduled in/out, employee/job references	More ops/scheduling than finance
Cash entries	GET /cashmgmt/v1/entries (/entries) (doc.toasttab.com)	Drawer events and cash	cashmgmt:read	One cash event	Essential if cash is material	Entry types include CASH_IN, CASH_OUT,	Needs config + employee joins

Domain	Endpoint / object	Purpose	Required scope	Grain	Warehouse suitability	Key fields / notes	Major limitations
		movement				NO_SALE, PAY_OUT, TIP_OUT, CLOSE_OUT_*, DRIVER_REIMBURSEMENT (doc.toasttab.com)	for interpretation
Cash deposits	GET /cashmgmt/v1/deposits (/deposits) (doc.toasttab.com)	Actual cash deposits	cashmgmt:read	One deposit entry	Essential if reconciling cash lifecycle	amount, employee, business date (doc.toasttab.com)	Separate from drawer entry events
Restaurant metadata	GET /restaurants/v1/restaurants/{restaurantGUID} (Toast API reference documentation)	Location metadata and closeout settings	restaurants:read	One restaurant	Required dimension	name, locationName, locationCode, timeZone, closeoutHour (doc.toasttab.com)	Needed to interpret businessDate correctly
Restaurant enu	GET /restaurants/v1/groups/{managementGroupGUI	Enumerate locations in a manage	restaurants:read	One restaurant GUID per row	Required for multi-	guid	Still must query each locatio

Domain	Endpoint / object	Purpose	Required scope	Grain	Warehouse suitability	Key fields / notes	Major limitations
meration	D}/restaurants (doc.toasttab.com)	ment group			location fanout		n separately
Menus	GET /menus and GET /metadata (doc.toasttab.com)	Menu and modifier master data	menus:read	Full resolved menu JSON	Required to normalize item/modifier hierarchy	Menu items, groups, modifiers, pricing, tax info (doc.toasttab.com)	Use /metadata or webhook to avoid unnecessary full pulls; Standard API supports Menus V2 only (Toast API reference documentation)
Revenue centers	GET /config/v2/revenueCenters (doc.toasttab.com)	Revenue-center dimension	config:read	One revenue center	Essential dimension	guid, name, description	Current-state dimension

Domain	Endpoint / object	Purpose	Required scope	Grain	Warehouse suitability	Key fields / notes	Major limitations
						(doc.toasttab.com)	
Sales categories	GET /config/v2/salesCategories (Toast API reference documentation)	Sales-category dimension	config:read	One sales category	Essential dimension	guid, name (Toast API reference documentation)	Current-state dimension
Dining options	GET /config/v2/diningOptions (doc.toasttab.com)	Dine-in / takeout / delivery behavior	config:read	One dining option	Essential dimension	guid, name, behavior, curbside (doc.toasttab.com)	Current-state dimension
Discounts	GET /config/v2/discounts (doc.toasttab.com)	Discount master data	config:read	One discount	Essential dimension	guid, name, active, type, percentage, amount (doc.toasttab.com)	Current-state dimension
Service charges	GET /config/v2/serviceCharges (Toast API reference documentation)	Service charge / auto-gratuity dimension	config:read	One service charge	Essential dimension	guid, name, amountType, amount, percent, criteria,	Current-state dimension

Domain	Endpoint / object	Purpose	Required scope	Grain	Warehouse suitability	Key fields / notes	Major limitations
						gratuity, taxable, applicable Taxes, destination (Toast API reference documentation)	
Tax rates	GET /config/v2/taxRates (doc.toasttab.com)	Tax dimension	config:read	One tax rate	Essential dimension	guid, name, rate, type, roundingType, conditionalTaxRates (doc.toasttab.com)	Current-state dimension
Alternate payment types	GET /config/v2/alternatePaymentTypes (doc.toasttab.com)	Map OTHER payments to named tenders	config:read	One alt payment type	Important for payment reporting	Alternate-payment descriptors (doc.toasttab.com)	Only for configured nonstandard tenders
Tip with	GET /config/v2/tipWithholding (doc.toasttab.com)	Percentage withhel	config:read	One restaurant config row	Important for labor/tip	enabled, percentage	Restaurant-level

Domain	Endpoint / object	Purpose	Required scope	Grain	Warehouse suitability	Key fields / notes	Major limitations
holding		d from CC tips			reporting	(doc.toasttab.com)	config only
Cash drawers	GET /config/v2/cashDrawers (doc.toasttab.com)	Cash drawer dimension	config:read	One drawer	Needed for cash event interpretation	Drawer master data (doc.toasttab.com)	Current-state dimension
No-sale reasons	GET /config/v2/noSaleReasons (doc.toasttab.com)	Dimension for no-sale cash events	config:read	One reason	Useful for ops controls	guid, name (doc.toasttab.com)	Current-state dimension
Payout reasons	GET /config/v2/payoutReasons (doc.toasttab.com)	Dimension for pay-out cash events	config:read	One reason	Useful for cash reporting	guid, name (doc.toasttab.com)	Current-state dimension

Objects inside the orders domain that should become separate warehouse tables

The Toast orders model is hierarchical. An Order contains one or more Check objects; checks contain Selection objects and Payment objects. Selections carry item, item-group, sales-category, modifiers, price, tax, void, and refund-related detail. Checks carry applied discounts, applied service charges, tax amounts, totals, payment status, and guest info when available. Payments carry amount, tip amount, type, refund info, void info, cashier/server references, cash drawer references, card metadata, and also a houseAccount reference when applicable. (doc.toasttab.com)

For finance and reporting, you should not leave the orders payload as one wide JSON table. The minimum normalized transactional breakdown is:

- order header
- check

- selection
- selection modifier
- check-level discount bridge
- selection-level discount bridge
- check-level service charge
- payment
- payment refund/void substructure
- applied taxes bridge at selection/service-charge level. This is a modeling recommendation based on the official Toast object structure. (doc.toasttab.com)

Specific coverage for the user's priority topics

Orders/checks/payments/items/modifiers/discounts/service charges/taxes/tips/refunds/voids are all covered in Standard API, but they are spread across the Order hierarchy and the separate /payments endpoint. Toast's sales-report cookbook explicitly points you to /ordersBulk for order-side metrics and /payments for payment-side metrics, including tips, refunds, and voids. (doc.toasttab.com)

Labor is covered by /labor/v1/timeEntries, /labor/v1/employees, and optionally /labor/v1/shifts. **Cash management** is covered by /entries and /deposits, plus config lookups such as /cashDrawers, /noSaleReasons, and /payoutReasons. **Restaurant/location metadata** is covered by the restaurants API. **Guests/customers** are available through the orders API, but are sparse and dining-option dependent. **House account / member attribution** is only partially visible: the official docs show a houseAccount reference on Payment, but in the Standard-API documentation reviewed here I did **not** find a corresponding Standard-API master endpoint for house accounts, so member attribution usually still needs an external system such as PeopleVine or another CRM/accounting map. (doc.toasttab.com)

3. Webhooks

Which Standard API webhooks exist

Toast's official webhook reference lists these event categories:

- Guest order fulfillment status
- Menus

- Orders
- Packaging preferences configuration
- Partners
- Restaurant availability
- Restaurant online ordering schedule
- Stock (doc.toasttab.com)

Which webhook events are useful for reporting ingestion

For a reporting warehouse, the useful webhook categories are:

- **Orders**: primary near-real-time change feed for orders. Toast recommends this over polling /ordersBulk for order updates. (doc.toasttab.com)
- **Menus**: change trigger for menu/mapping refresh; Toast says prefer the menus webhook and use /metadata as backup. (doc.toasttab.com)
- **Partners**: useful if your integration must stay aligned to connected restaurants/restaurant mappings over time. Toast recommends periodic Partners API polling even if you use the partners webhook. (doc.toasttab.com)

The other webhook categories are operationally useful but are not core finance/reporting feeds. (doc.toasttab.com)

Whether orders webhooks are enough for near-real-time ingestion

For order-change detection: yes. For a finance-grade warehouse: no. Toast recommends the orders webhook for order updates, but Toast's own warehouse cookbook still separately requires recurring retrieval of /payments, /timeEntries, /entries, /deposits, menu/configuration data, and restaurant metadata. So an orders webhook alone is not sufficient for finance, cash, labor, or reconciliation completeness. (doc.toasttab.com)

Recommended webhook + backfill architecture based on Toast docs

1. **Raw ingest all orders webhook payloads** into a bronze/raw zone, keyed by webhook event GUID for idempotence. Acknowledge within 2 seconds and process asynchronously. (doc.toasttab.com)
2. **Run scheduled /ordersBulk backfill/replay** by modifiedDate (startDate/endDate) to repair misses and guarantee completeness. Toast explicitly recommends

startDate/endDate for order retrieval by modification timestamp. (doc.toasttab.com)

3. **Pull /payments separately** by paidBusinessDate, refundBusinessDate, and voidBusinessDate, and do it at least six hours after closeoutHour. (doc.toasttab.com)
4. **Poll /timeEntries, /entries, /deposits** on a schedule. (doc.toasttab.com)
5. **Use menus webhook + /metadata backup**; only pull full /menus when stale. (doc.toasttab.com)
6. **Daily pull config endpoints with lastModified** where supported. Toast's reporting cookbooks explicitly recommend daily retrieval of the config objects used to interpret transactions. (doc.toasttab.com)

My implementation inference on top of Toast's guidance is to add overlap windows and deduplicate by natural keys plus latest modifiedDate. That overlap is not a Toast requirement, but it is the safest warehouse pattern given Toast's modified-timestamp retrieval model and webhook retry behavior. (doc.toasttab.com)

4. Standard API vs Analytics API

Topic	Standard API	Analytics API
Access model	Read-only, location-specific, Standard API credentials, no sandbox, GET-only for Standard access. (doc.toasttab.com)	Separate API family for management-group analytics requests. (doc.toasttab.com)
Subscription requirement	RMS Essentials or higher per location for Standard API access. (doc.toasttab.com)	RMS **Pro or higher** required. (doc.toasttab.com)
Scope	Domain-specific scopes like orders:read, config:read, labor:read, etc. (doc.toasttab.com)	enterprise-metrics:read for all analytics endpoints. (doc.toasttab.com)
Data grain	Transaction detail: orders, checks, selections, payments, time entries, cash events,	Aggregated/report-oriented datasets and report requests. (doc.toasttab.com)

Topic	Standard API	Analytics API
	config, menus. (Toast API reference documentation)	
Coverage	Full detail needed to build your own warehouse facts and dimensions. (doc.toasttab.com)	Adds aggregated sales, check reporting, labor reporting, menu reporting, payout reporting, guest payments, and restaurant-info at management-group scope. (doc.toasttab.com)
Payout views	No dedicated payout-reporting API in Standard docs; payment detail is transaction-level via /payments. (doc.toasttab.com)	Dedicated payout endpoints like /era/v1/payout/{timeRange}, /era/v1/payout/payments/{timeRange}, /era/v1/payout/sales-date/{timeRange}. (doc.toasttab.com)
Accounting suitability	Better base for finance-grade warehouse because it exposes the raw transaction detail you can model and reconcile yourself. This is an inference from the docs. (doc.toasttab.com)	Toast explicitly says analytics metrics do **not** adhere to GAAP/accounting standards and are for informational purposes. (doc.toasttab.com)
Is it enough alone?	**Usually yes for a production reporting warehouse**, if you normalize orders + payments + labor + cash + config + menus + restaurants, and if you solve club/member attribution externally where needed. This is an implementation inference based on Toast's warehouse cookbook and object coverage. (doc.toasttab.com)	Helpful add-on for management-group aggregate/payout analytics, but not a replacement for transaction-detail warehousing. (doc.toasttab.com)

5. Recommended warehouse design

Recommended fact tables

This is the normalized model I would use for Toast in a finance/ops warehouse:

Table	Grain	Key natural identifiers	Main measures / payload
f_order	One order	restaurant_guid + order_guid	source, dining option, promised/opened/closed/modified dates, order status flags, revenue center, server
f_check	One check	restaurant_guid + check_guid	amount, tax amount, total amount, payment status, tab name, tax exempt, guest reference
f_selection	One menu-item selection	restaurant_guid + selection_guid	quantity, pre-discount price, price, tax, void flags/dates, fulfillment status, item refs
f_selection_modifier	One applied modifier option	restaurant_guid + selection_guid + modifier_position	modifier item refs, incremental price/tax
f_payment	One payment	restaurant_guid + payment_guid	amount, tip amount, tender type, server, cash drawer, refund/void subfields, card metadata, house-account ref
f_check_service_charge	One applied service charge instance	restaurant_guid + check_guid + service_charge_guid + payment_guid?	charge amount, gratuity flag, taxable flag, refund details
f_check_discount_bridge	One check-level	restaurant_guid + check_guid +	discount amount/value

Table	Grain	Key natural identifiers	Main measures / payload
	applied discount instance	discount_guid + ordinal	
f_selection_discount_bridge	One selection-level applied discount instance	restaurant_guid + selection_guid + discount_guid + ordinal	discount amount/value
f_selection_tax_bridge	One tax application	restaurant_guid + selection_guid + tax_guid + ordinal	tax amount/rate snapshot
f_time_entry	One time entry	restaurant_guid + time_entry_guid	in/out times, hours, declared cash tips, non-cash tips, deleted flag
f_break	One break inside a time entry	restaurant_guid + time_entry_guid + break_ordinal	break type, paid flag, in/out, missed
f_cash_entry	One cash event	restaurant_guid + cash_entry_guid	amount, entry type, employee, drawer, reason
f_cash_deposit	One cash deposit	restaurant_guid + deposit_guid	amount, employee, business date

This fact design follows Toast's documented grains: order → check → selection/payment in the orders API, one row per payment in /payments, one row per time entry in labor, and one row per cash event/deposit in cash management. (doc.toasttab.com)

Recommended dimension tables

Use at minimum:

- d_restaurant
- d_employee

- d_revenue_center
- d_dining_option
- d_sales_category
- d_discount
- d_service_charge_type
- d_tax_rate
- d_alt_payment_type
- d_cash_drawer
- d_no_sale_reason
- d_payout_reason
- d_menu_item
- d_menu_group
- d_modifier_group
- d_modifier_option
- d_guest as a **sparse** dimension only for orders where customer data exists. Toast documents that customer information is mainly for takeout/delivery and is null for in-store orders in the CRM guidance. (doc.toasttab.com)

Recommended keys and relationships

Use Toast GUIDs as **natural business keys** and add warehouse surrogate keys for BI/star-schema joins. Always include:

- restaurant_guid
- the Toast entity GUID
- business_date
- UTC timestamp fields (createdDate, modifiedDate, etc.)
- ingested_at_utc
- record_source (orders_webhook, ordersBulk, payments_api, etc.)

That key pattern is my recommendation, but it is directly motivated by Toast's location-specific request model, GUID-based entity identifiers, and the centrality of businessDate, modifiedDate, and closeoutHour. (doc.toasttab.com)

Suggested raw / staging / final layers

Raw/Bronze: store webhook bodies and full API JSON exactly as received.
Staging/Silver: flatten and deduplicate by natural key + latest modifiedDate; separate nested arrays into child tables.
Final/Gold: finance/reporting facts and dimensions aligned to Power BI semantic models.
Microsoft's Fabric guidance recommends medallion architecture in OneLake, with bronze/raw, silver/enriched, and gold/curated layers. (Microsoft Learn)

6. Fabric implementation mapping

Warehouse or Lakehouse as landing zone?

For **landing**, I recommend **Fabric Lakehouse**, not Warehouse. Microsoft's decision guide says Lakehouse is the better fit for **structured + semi-structured data** and for Spark-oriented development, while Warehouse is the better fit for **structured-only**, T-SQL-heavy, and multi-table-transaction use cases. Toast order/webhook payloads are nested and semi-structured JSON, which maps naturally to Lakehouse bronze/silver processing. (Microsoft Learn)

For **serving**, you can either:

- keep curated Delta tables in Lakehouse and expose them through the SQL analytics endpoint, or
- publish a Fabric Warehouse gold layer if your team prefers tightly managed T-SQL star-schema curation.

Microsoft explicitly documents that Lakehouse automatically exposes Delta tables through a SQL analytics endpoint, and also documents that Warehouse is the stronger choice when you want SQL-engine multi-table transactional guarantees. (Microsoft Learn)

Recommended Fabric layout

- **Bronze Lakehouse**: raw webhook JSON, raw /ordersBulk, raw /payments, raw /timeEntries, raw /entries, raw /deposits, raw config snapshots, raw menus snapshots.

- **Silver Lakehouse**: normalized Delta tables for orders, checks, selections, modifiers, payments, refunds, voids, cash entries, deposits, time entries, breaks, config dims, menu dims.
- **Gold**: either curated Warehouse marts or curated Lakehouse Delta tables consumed by Power BI Direct Lake. Microsoft documents Direct Lake as especially well-suited for Fabric lake-centric architectures and for gold analytics layers over Delta tables. (Microsoft Learn)

Which queries should exist first

Start with these extraction/query groups, in this order:

1. restaurants
2. menus metadata
3. menus
4. config dimensions
5. ordersBulk
6. payments
7. employees
8. timeEntries
9. cash entries
10. cash deposits

That order matches Toast's own warehouse guidance: determine closeoutHour, load menus/configuration, then retrieve transactional entities. (doc.toasttab.com)

Which tables should be landed first

First landing set:

- raw_orders_bulk
- raw_payments
- raw_time_entries
- raw_cash_entries
- raw_cash_deposits

- raw_restaurants
- raw_menus_metadata
- raw_menus
- raw_config_*

First normalized set:

- stg_order
- stg_check
- stg_selection
- stg_payment
- stg_employee
- stg_time_entry
- stg_cash_entry
- stg_cash_deposit
- config/menu dimensions

That is the minimum viable production slice for finance, labor, and operational reporting from Toast's documented data model. (doc.toasttab.com)

Which entities should be split into separate tables

You should split these immediately:

- Order from Check
- Check from Selection
- Selection from modifier rows
- Check from Payment
- discount bridges from core entities
- service charge instances from checks
- tax applications from selections/service charges
- time-entry breaks from time entries

That split is necessary because Toast's nested objects have different grains and update patterns. (doc.toasttab.com)

Recommended incremental refresh / date-window strategy

From Toast docs:

- **Orders**: use /ordersBulk with startDate/endDate on modifiedDate; use webhook as primary change feed. (doc.toasttab.com)
- **Payments**: use /payments by paidBusinessDate, refundBusinessDate, voidBusinessDate; retrieve at least six hours after closeoutHour. (doc.toasttab.com)
- **Time entries**: use modifiedStartDate/modifiedEndDate for deltas. (doc.toasttab.com)
- **Cash**: use businessDate daily for /entries and /deposits. (doc.toasttab.com)
- **Config**: use lastModified on config endpoints where supported. (doc.toasttab.com)
- **Menus**: use menus webhook or /metadata; only reload /menus when stale. (doc.toasttab.com)

My recommended operational windows on top of Toast's rules are:

- orders replay every 15–30 minutes with a 30–60 minute overlap
- payments replay for the current business date plus the prior 2 business dates
- nightly "close" reconciliation run after closeoutHour + 6h
- weekly repair job for the trailing 14 days

Those overlap windows are my implementation recommendation, not a Toast requirement. They are intended to protect against late modifications, webhook retries, and cross-business-date timing. (doc.toasttab.com)

Recommended orchestration approach

Use **Fabric Data Factory pipelines** to orchestrate the workflow. Microsoft documents Fabric pipelines as the orchestration layer for scheduled data movement and transformation workflows. For this use case, one pipeline should run the scheduled API pulls and downstream normalization, while the webhook handler writes raw events immediately into bronze storage. (Microsoft Learn)

7. Gaps / risks

1. **House account / member attribution gap**: the official docs show houseAccount on Payment, but in the Standard API docs reviewed here I did not find a documented Standard-API master endpoint for house-account entities. For a membership club, that means Toast alone may not provide the member/account dimension you need; PeopleVine or another membership system will likely still be required for authoritative member attribution. (Toast API reference documentation)
2. **Guest data is sparse and not reliable for dine-in identity resolution**: Toast's CRM guidance says Customer appears only where dining-option behavior is TAKE_OUT or DELIVERY, and the customer object is null for in-store orders. That is a major limitation for club/member analytics if the member identity does not flow in from another system. (doc.toasttab.com)
3. **Payments must be modeled separately from orders**: Toast's own cookbooks treat /ordersBulk and /payments as separate retrievals. If you try to do finance reporting from orders alone, you will miss the payment-specific business-date logic for voids/refunds and the documented recommendation to wait six hours after closeout to pick up all credit card payments and fees. (doc.toasttab.com)
4. **businessDate is not the same as event timestamp**: businessDate is driven by restaurant closeoutHour, and unpaid orders can stay open across business dates. That creates reconciliation complexity unless you store both raw timestamps and business-date keys. (doc.toasttab.com)
5. **Time entries are mutable until clock-out**: Toast documents that active time entries can continue changing while the employee is still working. That means labor facts need delta logic and should not be treated as final until the shift closes. (doc.toasttab.com)
6. **Standard API is location-specific**: for multi-location clubs or hospitality groups, every extraction must fan out by location GUID. The management-group endpoint helps enumerate locations, but the actual pulls remain per location. (doc.toasttab.com)
7. **Menus V3 is not part of Standard API access**: Standard access supports Menus V2 only. (doc.toasttab.com)
8. **No sandbox in Standard API access**: this matters for deployment/testing controls. (doc.toasttab.com)

8. Deliverables

A. Recommended Toast endpoints to implement first

Priority	Endpoint	Why first
1	/restaurants/v1/restaurants/{restaurantGUID} (doc.toasttab.com)	Required for closeoutHour, time zone, and location dimension
2	/menus + /metadata (doc.toasttab.com)	Required to decode items, modifiers, pricing/tax structure
3	/config/v2/revenueCenters, /salesCategories, /diningOptions, /discounts, /serviceCharges, /taxRates, /alternatePaymentTypes, /tipWithholding (doc.toasttab.com)	Required to interpret transactions
4	/orders/v2/ordersBulk (Toast API reference documentation)	Main transactional backfill and replay source
5	Orders webhook (doc.toasttab.com)	Near-real-time order change capture
6	/payments (doc.toasttab.com)	Finance-grade payment/refund/void/tip detail
7	/labor/v1/employees (doc.toasttab.com)	Employee/server dimension
8	/labor/v1/timeEntries (Toast API reference documentation)	Labor transaction facts
9	/cashmgmt/v1/entries and /cashmgmt/v1/deposits (doc.toasttab.com)	Cash controls and closeout reporting
10	/config/v2/cashDrawers, /noSaleReasons, /payoutReasons (doc.toasttab.com)	Decode cash events

B. Required scopes table

Scope	Needed for
orders:read (doc.toasttab.com)	/ordersBulk, /orders/{guid}, /payments, payment/order reporting
config:read (doc.toasttab.com)	revenue centers, dining options, discounts, service charges, taxes, alt payments, tip withholding, cash reasons
menus:read (doc.toasttab.com)	menus and modifier hierarchy
restaurants:read (doc.toasttab.com)	location metadata and closeout hour
labor:read (doc.toasttab.com)	time entries, shifts
labor.employees:read (doc.toasttab.com)	employees/server dimension
cashmgmt:read (doc.toasttab.com)	cash entries and deposits
guest.pi:read (optional but often needed) (doc.toasttab.com)	guest name/phone/email and curbside info
delivery_info.address:read (optional) (doc.toasttab.com)	delivery addresses and notes

C. Draft query build plan

Bootstrap

1. Get location list and restaurant metadata. (doc.toasttab.com)
2. Pull /metadata, then /menus. (doc.toasttab.com)
3. Pull all core config dimensions. (doc.toasttab.com)
4. Backfill historical orders with /ordersBulk in one-month windows, spaced 5–10 seconds apart; Toast recommends about twelve weeks for initial history in its reporting cookbooks. (doc.toasttab.com)
5. Backfill payments by business date using paidBusinessDate, refundBusinessDate, and voidBusinessDate. (doc.toasttab.com)
6. Backfill employees/time entries/cash entries/deposits. (doc.toasttab.com)

Ongoing

1. Orders webhook to bronze. (doc.toasttab.com)
2. /ordersBulk replay by modified window. (doc.toasttab.com)
3. /payments replay by business-date statuses, especially after closeout+6h. (doc.toasttab.com)
4. Daily config lastModified pulls. (doc.toasttab.com)
5. Menus webhook + /metadata backup every ~30 minutes. Toast explicitly recommends /metadata as the lightweight stale check and the menus webhook as preferred. (doc.toasttab.com)

D. Draft table schema plan

Bronze/raw

- raw_toast_orders_webhook
- raw_toast_orders_bulk
- raw_toast_payments
- raw_toast_time_entries
- raw_toast_cash_entries
- raw_toast_cash_deposits
- raw_toast_restaurants
- raw_toast_menus
- raw_toast_menus_metadata
- raw_toast_config_<entity>

Silver/staging

- stg_order
- stg_check
- stg_selection
- stg_selection_modifier
- stg_payment

- stg_payment_refund_void
- stg_check_service_charge
- stg_check_discount
- stg_selection_discount
- stg_selection_tax
- stg_employee
- stg_time_entry
- stg_time_entry_break
- stg_cash_entry
- stg_cash_deposit
- stg_dim_* for all config/menu/restaurant dimensions

Gold/final

- fact_sales_check
- fact_sales_selection
- fact_payments
- fact_labor_time_entry
- fact_cash_activity
- dim_restaurant
- dim_employee
- dim_menu_item
- dim_revenue_center
- dim_dining_option
- dim_discount
- dim_service_charge
- dim_tax_rate
- dim_alt_payment_type

- dim_guest_sparse

That schema plan is my recommendation, but it is directly aligned to Toast's documented order, payment, labor, cash, and configuration objects. (doc.toasttab.com)

E. Phased implementation roadmap

Phase 1: Proof of concept

- One location
- restaurants + menus + config + /ordersBulk + /payments
- build order/check/selection/payment model
- validate gross sales, tax, service charges, discounts, tips, refunds, voids against Toast UI/reports. Toast's sales and warehouse cookbooks define these metrics directly from Standard API fields. (doc.toasttab.com)

Phase 2: Operational completeness

- add employees + time entries
- add cash entries + deposits
- add orders webhook
- implement daily config refresh and menu stale detection. (doc.toasttab.com)

Phase 3: Production hardening

- multi-location fanout
- replay windows and idempotent merge logic
- closeout+6h payment reconciliation job
- raw retention + lineage + error handling for webhook retries/pauses. (doc.toasttab.com)

Phase 4: Club-specific enrichment

- join external membership/house-account systems
- add PeopleVine/member-account bridge tables
- build member spend, house account, and dues-adjacent operational reporting outside native Toast guest limitations. This phase is necessary because Toast's

documented Standard API guest/member coverage is incomplete for dine-in/member attribution. (doc.toasttab.com)

The shortest executive answer is: **implement Toast Standard API first, not Analytics API, as your warehouse backbone**; use **orders webhook + /ordersBulk replay + /payments reconciliation + daily config/menu/labor/cash pulls**; land raw data in a **Fabric Lakehouse**, normalize to Delta tables, and expose curated finance marts to Power BI through **Direct Lake** or a gold Warehouse layer. That architecture is the closest fit to both Toast's official warehouse guidance and Microsoft Fabric's official Lakehouse/Warehouse guidance. (doc.toasttab.com)